

Dragging and dropping objects (Chapter 14)

This section contains additional information that is not present in the book.

Dragging a Canvas subject

Most SVG dragging applications need no more setup than the examples shown. For Canvas, however, you must identify the dragged object as an area by configuring a *custom subject*. In the following example, a grid and nine squares are rendered. This snippet configures canvas, data and scale:

```
const dim = {w: 800, h: 800};
const canvas = d3.select("body")
    .append("canvas").attr("width", dim.w).attr("height", dim.h);
const ctx = canvas.node().getContext("2d");
const data = [{color: '#e41a1c', x:0, y:0}, /* ... */, {color: '#999999', x:2, y:2}];
const scale = d3.scaleLinear().range([102,100+dim.w]).domain([0,data.length-1]);
```

The data will be used to render the squares on the canvas. The `render()` function places each square in its position, according to the current data. It will be called every time the data changes:

```
const side = 96; // side of the square
function render() {
    ctx.clearRect(0, 0, dim.w, dim.h);
    data.forEach(d => {
        ctx.fillStyle = d.color;
        ctx.fillRect(scale(d.x), scale(d.y), side, side);
    });
}
```

In a Canvas it is necessary to check if the pointer is clicked within the square. The `getSubject()` function finds the index of topmost object that contains the clicked point, then returns the corresponding object. The `points()` function was created to obtain the square as a list of points, so it could be used in `d3.polygonContains()` (from the *d3-polygon* module):

```
const points = (x,y) => [[x,y],[x+side,y],[x+side,y+side],[x,y+side]];
function getSubject(evt) {
    const index = data.findLastIndex(d => {
        const polygon = points(scale(d.x), scale(d.y));
        return d3.polygonContains(polygon, d3.pointer(evt));
    });
}
```

```

    return index >= 0 ? data[index] : null;
  }

```

The `{x,y,color}` object returned from the function above is used to configure the subject in the drag function, which is the dragged object. While the drag event occurs, the object's x and y coordinates are updated, and the canvas is rendered again.

```

const drag = d3.drag()
    .container(canvas)           // default if you call drag from canvas
    .subject(evt => getSubject(evt))
    .on("drag", function(evt) {
        evt.subject.x = scale.invert(evt.dx + scale(evt.subject.x));
        evt.subject.y = scale.invert(evt.dy + scale(evt.subject.y));
        render();
    })

```

To show the squares, call `render()` and then call the drag behavior from the canvas selection:

```

render();
canvas.call(drag);

```

See the full code in `Drag/10-canvas-subject.html`. The screenshots in *Figure 1* display the result before and during a dragging operation.

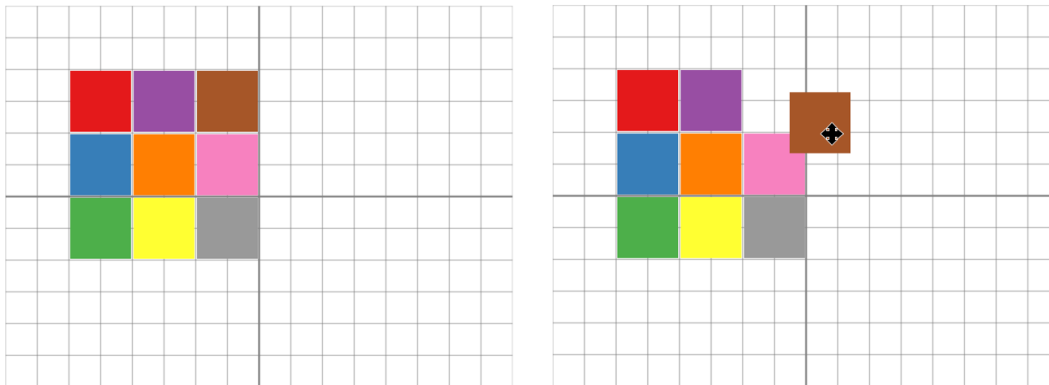


Figure 1 – Dragging an object using Canvas. Code: *Drag/10-canvas-subject.html*.